ClearCut: Rule-Based Spoiler-Free Rewriting of Movie and Series Reviews

Natural Language Processing Project Report

Tejeshwar | [Roll number, programme, institution] | Submitted to: [Professor, department] | 23 October 2026

Code: https://github.com/tejeshwar20/clearcut

Abstract

Reviews often reveal plot twists, deaths and endings. ClearCut is a web application that takes a pasted English review and returns the same review with only the spoiler sentences rewritten; every other sentence is kept word for word. It is a rule-based NLP pipeline: spaCy dependency parsing decides how much of a sentence to remove, curated pattern sets detect plot disclosures, and a rewriting step keeps the reviewer's reaction while replacing the revealed fact with a neutral phrase. A final check withholds the output if any disclosure remains. On developer-written tests the system leaked 0 of 25 spoilers, changed 0 of 20 harmless sentences, kept 48 of 48 harmless sentences around a spoiler, and responded in about 20 ms (short) to 130 ms (300 words). These tests are not an independent benchmark.

1. Introduction

Readers who want a reviewer's opinion but not the plot have few options. Sites offer spoiler tags, and extensions blur keyword matches, but both hide the opinion along with the plot. ClearCut instead rewrites the spoiler and keeps the opinion. For example, “Great visuals. It was all a simulation the whole time, which blew my mind.” becomes “Great visuals. The big surprise blew my mind.” The reviewer's reaction survives and the revealed fact does not.

2. Problem statement

Given a free-text review, produce a version that **(a)** discloses no plot information, **(b)** keeps every sentence that does not disclose plot exactly as written, and **(c)** keeps the reviewer's reaction wherever this is safe. Existing spoiler-detection research mostly labels text as spoiler / not spoiler; this project goes further and rewrites the text, under the strict constraint that harmless text is never altered.

3. Solution approach

Model. There is no trained model. A transparent rule-based pipeline was chosen because no labelled spoiler data was available, because “other sentences stay identical” is easy to guarantee with rules but not with a generative model, and because every decision can be traced to a named pattern. It runs offline and needs no API key.

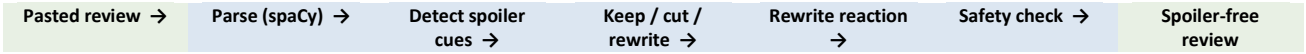


Figure 1. *Processing pipeline. Sentences with no spoiler cue skip straight through unchanged.*

Pipeline (spaCy 3.8 `en_core_web_sm`, regular expressions, VADER 3.3.2, Flask/Waitress):

- **Parse:** split into sentences; obtain part-of-speech tags, lemmas and the dependency tree.
- **Detect:** match five cue sets – events (deaths, betrayals), reveals (“turns out”, “the climax”), hidden reveals (identity and family reveals, “all along”, dream or simulation twists, “Spoiler:”), outcomes, and character fates. Genre labels such as “revenge thriller” are exempt.
- **Scope:** no cue → sentence kept unchanged. Cue in a subordinate clause → only that clause is removed. Cue in the main clause or a severe reveal → the sentence is rewritten.
- **Rewrite:** keep the reviewer's reaction and point it at a neutral phrase (“the big surprise”); if no opinion can be kept, use a neutral line chosen by spoiler type (ending, climax, twist, fate). Lines are never repeated in one review, and consecutive spoilers are merged into one line.
- **Safety check:** the output is re-scanned for disclosure words and long copied runs of the source; if any are found, nothing is shown (the system fails closed).

Design decisions. A sentence-embedding “semantic” layer was built and then removed because it altered harmless sentences and raised memory from about 238 MB to 398 MB. An estimated review score was also removed, because it was only a guess from wording and not part of the task.

4. Experimental results

A script (`artifacts/measure.py`) runs fixed test sets through the live pipeline. Each spoiler case is a harmless opening sentence followed by a spoiler; the test lists terms that must not appear in the output. Cases were written by the developer, so the results show the rules behave as designed, not accuracy on unseen reviews.

Table 1. *Measured results.*

Measure	Result
Spoiler sentences tested / leaked into output	25 / **0**
Harmless sentences tested / changed or blocked	20 / **0**
Mixed reviews (harmless + spoiler + harmless): harmless sentences kept intact	**48 of 48**
Whole review wrongly blocked	0
Latency, short review (about 2 sentences): median / 95th percentile	20 ms / 26 ms
Latency, 297-word review: median / maximum	127 ms / 170 ms
Automated tests passing	212 of 212
Earlier aspect-sentiment component (`evaluate.py`, 26 synthetic examples)	spoiler accuracy 0.92; sentiment accuracy 0.89

Table 2. *Worked examples (actual outputs of the final system).*

Input	Output
The acting is superb. Darth Vader is Luke's father, and that scene still gives me chills. The pacing is tight.	The acting is superb. The big surprise still gives me chills. The pacing is tight.
Fun ride. Turns out the mentor was the villain, which I didn't see coming.	Fun ride. I didn't see the big surprise coming.
The cast is superb and the songs are catchy. In the end the hero dies saving his village. The cinematography is gorgeous.	The cast is superb and the songs are catchy. The ending is best experienced without knowing what's coming. The cinematography is gorgeous.
The visual effects are stunning, but I hated that the main character dies at the end.	The visual effects are stunning.
A tense, gripping war drama with strong performances.	(unchanged: no spoiler)

Finding. The first run of the 25-case set exposed one leak ("Amy faked her own disappearance to frame Nick"), although an earlier 16-case probe had shown none. The patterns were extended, a test was added, and the re-run gave 0 leaks. A real failure was also fixed: a pasted plot summary of *Devara: Part 1* was blocked entirely because a rewrite produced "The big surprise is dead"; it now keeps the setup verbatim and replaces only the twist and ending.

Limitations. Rules cover only anticipated phrasings, so an unusual spoiler can pass; replacement lines are generic by design; only English is supported; and accuracy on real reviews is unknown until tested on an independent, labelled set (planned: 100 or more reviews, two annotators).

5. Data source

The system uses **no training data**. Test and evaluation data are as follows.

- **Test reviews and spoiler sentences:** written by the developer, based on the plots of well-known films (for example *The Sixth Sense*, *Fight Club*, *Parasite*, *Gone Girl*) and on reviews that failed during use (*Jailer*, *Irumudi*, *Devara*). They live in `tests/` and `artifacts/measure.py`.
- **Evaluation file:** `data/evaluation_examples.json` holds 26 short synthetic sentences used by `evaluate.py`; it is a sanity check, not a benchmark.
- **Libraries and lexicons:** spaCy `en_core_web_sm` 3.8.0 (parser) and the VADER sentiment lexicon (published rule-based model, Hutto and Gilbert, 2014), both used unmodified except for a small added review vocabulary.
- **User input:** pasted reviews are processed in memory and are not stored by the pasted-review path.

Source code, tests and the full 14-section report: https://github.com/tejeshwar20/clearcut